

Лабораторная работа №5
«Интерполяция функции»
Вариант №11

Выполнил:
Студент группы Р3208
Решетников С.Е.

Преподаватели:
Рыбаков С.Д.

Дата сдачи: 1 мая
Санкт-Петербург, 2026

1. Цель работы

Решить задачу интерполяции: построить интерполяционные многочлены и найти значения функции при заданных значениях аргумента, отличных от узловых точек.

2. Порядок выполнения работы

- выбрать таблицу значений функции для варианта 11;
- построить таблицу конечных разностей;
- вычислить значение функции в точке X_1 по первой или второй формуле Ньютона;
- вычислить значение функции в точке X_2 по первой или второй формуле Гаусса;
- вычислить значения функции в точках X_1 и X_2 с помощью многочлена Лагранжа;
- для дополнительного задания реализовать схемы Стирлинга и Бесселя;
- реализовать программу с вводом данных вручную, из файла и через табулирование функции;
- построить график узлов интерполяции и интерполяционных многочленов.

3. Рабочие формулы

3.1. Многочлен Лагранжа

Для узлов $x_0, x_1, \dots, x_n$ многочлен Лагранжа имеет вид:

$$L_{n(x)} = \sum_{i=0}^n y_i l_{i(x)}, \quad l_{i(x)} = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

3.2. Конечные разности

Для равноотстоящих узлов $x_i - x_{i-1} = h$ конечные разности считаются по формулам:

$$\Delta y_i = y_{i+1} - y_i, \quad \Delta^k y_i = \Delta^{k-1} y_{i+1} - \Delta^{k-1} y_i$$

3.3. Формула Ньютона для интерполирования вперед

Используется в левой части таблицы. Пусть $t = \frac{x-x_0}{h}$. Тогда:

$$N_{n(x)} = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \dots + \frac{t(t-1)\dots(t-n+1)}{n!}\Delta^n y_0$$

3.4. Формула Ньютона для интерполирования назад

Используется в правой части таблицы. Пусть $t = \frac{x-x_n}{h}$. Тогда:

$$N_{n(x)} = y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!}\Delta^2 y_{n-2} + \dots + \frac{t(t+1)\dots(t+n-1)}{n!}\Delta^n y_0$$

3.5. Первая формула Гаусса

Для точки справа от центрального узла a используется первая формула Гаусса. Пусть $t = \frac{x-a}{h}$:

$$P_{n(x)} = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-1} + \dots$$

3.6. Вторая формула Гаусса

Для точки слева от центрального узла a используется вторая формула Гаусса:

$$P_{n(x)} = y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \dots$$

3.7. Формула Стирлинга

Формула Стирлинга является средним арифметическим первой и второй формул Гаусса. Она применяется при значениях t , близких к нулю, и строится по нечётному числу узлов:

$$S_{n(x)} = y_0 + t \frac{\Delta y_{-1} + \Delta y_0}{2} + \frac{t^2}{2!} \Delta^2 y_{-1} + \frac{t(t^2 - 1)}{3!} \frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2} + \dots$$

3.8. Формула Бесселя

Формула Бесселя применяется при значениях t , близких к 0.5, и строится по чётному числу узлов:

$$B_{n(x)} = \frac{y_0 + y_1}{2} + \left(t - \frac{1}{2}\right) \Delta y_0 + \frac{t(t-1)}{2!} \frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} + \frac{\left(t - \frac{1}{2}\right)t(t-1)}{3!} \Delta^3 y_{-1} + \dots$$

4. Вычислительная часть

4.1. Исходные данные

Вариант 11 соответствует таблице 1.1 из задания.

$$X_1 = 0.255, \quad X_2 = 0.405, \quad h = 0.05$$

i	0	1	2	3	4	5	6
x_i	0.25	0.30	0.35	0.40	0.45	0.50	0.55
y_i	1.2557	2.1764	3.1218	4.0482	5.9875	6.9195	7.8359

4.2. Таблица конечных разностей

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0	0.25	1.2557	0.9207	0.0247	-0.0437	1.0756	-4.1277	10.1917
1	0.30	2.1764	0.9454	-0.0190	1.0319	-3.0521	6.0640	
2	0.35	3.1218	0.9264	1.0129	-2.0202	3.0119		
3	0.40	4.0482	1.9393	-1.0073	0.9917			
4	0.45	5.9875	0.9320	-0.0156				
5	0.50	6.9195	0.9164					
6	0.55	7.8359						

4.3. Вычисление значения в точке $X_1 = 0.255$ по формуле Ньютона

Точка $X_1 = 0.255$ находится в левой части таблицы, поэтому используется первая формула Ньютона.

$$t = \frac{x - x_0}{h} = \frac{0.255 - 0.25}{0.05} = 0.1$$

$$N_6(0.255) = 1.2557 + 0.1 \cdot 0.9207 + (-0.0437) \cdot 0.0247 + 0.0285 \cdot (-0.0437) + (-0.0206625) \cdot 1.0756 + 0.01611675 \cdot (-4.1277) + (-0.01316201) \cdot 10.1917 = 1.12252007$$

Получено:

$$f(0.255) \approx 1.12252007$$

4.4. Вычисление значения в точке $X_2 = 0.405$ по формуле Гаусса

Точку $X_2 = 0.405$ удобно считать относительно центрального узла $a = 0.40$. Так как $0.405 > 0.40$, используется первая формула Гаусса.

$$t = \frac{x - a}{h} = \frac{0.405 - 0.40}{0.05} = 0.1$$

Для центральной таблицы:

$$y_0 = 4.0482, \quad \Delta y_0 = 1.9393, \quad \Delta^2 y_{-1} = 1.0129 \quad \Delta^3 y_{-1} = -2.0202 \quad \Delta^4 y_{-2} = -3.0521 \\ \Delta^5 y_{-2} = 6.0640 \quad \Delta^6 y_{-3} = 10.1917$$

$$P_6(0.405) = 4.0482 + 0.1 \cdot 1.9393 + (-0.045) \cdot 1.0129 + (-0.0165) \cdot (-2.0202) + \\ 0.0078375 \cdot (-3.0521) + 0.00329175 \cdot 6.0640 + (-0.00159101) \cdot 10.1917 = 4.20970802$$

Получено:

$$f(0.405) \approx 4.20970802$$

4.5. Вычисление по многочлену Лагранжа

Для проверки вычислены значения полного многочлена Лагранжа шестой степени:

$$L_6(0.255) = 1.12252007 \quad L_6(0.405) = 4.20970802$$

Результаты совпали со значениями, полученными по формулам Ньютона и Гаусса.

4.6. Дополнительные схемы интерполяции

Для дополнительной части были реализованы схемы Стирлинга и Бесселя.

Формула Стирлинга строится по нечётному числу узлов. Для варианта 11 использованы все 7 узлов таблицы, поэтому результат совпал со значением полного интерполяционного многочлена:

$$S_6(0.255) = 1.12252007 \quad S_6(0.405) = 4.20970802$$

Формула Бесселя строится по чётному числу узлов и практически применяется при $0.25 \leq t \leq 0.75$. Для имеющейся таблицы программа выбирает максимально симметричный шеститочечный набор вокруг пары центральных узлов:

$$B(0.255) = 1.25666336 \quad B(0.405) = 4.22592314$$

Отличие значений Бесселя от полного многочлена объясняется тем, что схема Бесселя в данном наборе использует чётное число узлов и не включает разность шестого порядка.

5. Программная реализация

В программу добавлены два класса для дополнительной части:

- StirlingInterpolator

вычисляет значение по формуле Стирлинга;
- BesselInterpolator

вычисляет значение по формуле Бесселя.

5.1. Листинг программы

Частичный листинг программы

```

from utils.dataset import InterpolationDataset
from utils.differences import FiniteDifferenceTable
from .InterpolationResult import InterpolationResult
from math import factorial, prod

class GaussInterpolator:
    def __init__(self, dataset: InterpolationDataset, differences: FiniteDifferenceTable) -> None:
        self.dataset = dataset
        self.differences = differences
        if dataset.step is None:
            raise ValueError("формула Гаусса требует равноотстоящие узлы")
        if len(dataset.x) < 3:
            raise ValueError("для формулы Гаусса нужно не менее трех узлов")

    def evaluate(self, x_value: float) -> InterpolationResult:
        center = self._center_index(x_value)
        if x_value >= self.dataset.x[center]:
            return self.evaluate_first(x_value, center)
        return self.evaluate_second(x_value, center)

    def evaluate_first(self, x_value: float, center: int | None = None) -> InterpolationResult:
        center = self._center_index(x_value) if center is None else center
        h = self.dataset.step
        assert h is not None
        t = (x_value - self.dataset.x[center]) / h
        total = self.dataset.y[center]
        terms = [f"y0={total:.8f}"]
        for order in range(1, len(self.dataset.x)):
            diff_index = center - order // 2
            if not self._has_difference(order, diff_index):
                break
            shifts = range((order - 1) // 2, -(order // 2) - 1, -1)
            coefficient = prod(t + shift for shift in shifts) / factorial(order)
            difference = self.differences.value(order, diff_index)
            total += coefficient * difference
            terms.append(f"k={order}: {coefficient:.8f}*{difference:.8f}")
        return InterpolationResult("первая формула Гаусса", total, f"a=x{center}, t={t:.8f}; " + "; ".join(terms))

    def evaluate_second(self, x_value: float, center: int | None = None) -> InterpolationResult:
        center = self._center_index(x_value) if center is None else center
        h = self.dataset.step
        assert h is not None
        t = (x_value - self.dataset.x[center]) / h
        total = self.dataset.y[center]
        terms = [f"y0={total:.8f}"]
        for order in range(1, len(self.dataset.x)):
            diff_index = center - (order + 1) // 2
            if not self._has_difference(order, diff_index):
                break
            shifts = range(order // 2, -(order - 1) // 2 - 1, -1)
            coefficient = prod(t + shift for shift in shifts) / factorial(order)
            difference = self.differences.value(order, diff_index)
            total += coefficient * difference
            terms.append(f"k={order}: {coefficient:.8f}*{difference:.8f}")
        return InterpolationResult("вторая формула Гаусса", total, f"a=x{center}, t={t:.8f}; " + "; ".join(terms))

    def _center_index(self, x_value: float) -> int:
        return len(self.dataset.x) // 2

    def _has_difference(self, order: int, index: int) -> bool:
        return 0 <= order < len(self.differences.levels) and 0 <= index < len(self.differences.levels[order])

```

```

from utils.dataset import InterpolationDataset
from .InterpolationResult import InterpolationResult

class LagrangeInterpolator:
    def __init__(self, dataset: InterpolationDataset) -> None:
        self.dataset = dataset

    def evaluate(self, x_value: float) -> InterpolationResult:
        total = 0.0
        terms: list[str] = []
        for i, (x_i, y_i) in enumerate(zip(self.dataset.x, self.dataset.y)):
            basis = 1.0
            for j, x_j in enumerate(self.dataset.x):
                if i != j:
                    basis *= (x_value - x_j) / (x_i - x_j)
            contribution = y_i * basis
            total += contribution
            terms.append(f"y{i}={contribution:.8f}")
        return InterpolationResult("многочлен Лагранжа", total, "; ".join(terms))

```

```

from utils.dataset import InterpolationDataset
from utils.differences import FiniteDifferenceTable
from .InterpolationResult import InterpolationResult
from math import factorial, prod

class NewtonInterpolator:
    def __init__(self, dataset: InterpolationDataset, differences: FiniteDifferenceTable) -> None:
        self.dataset = dataset
        self.differences = differences
        if dataset.step is None:
            raise ValueError("формула Ньютона с конечными разностями требует равноотстоящие узлы")

    def evaluate(self, x_value: float) -> InterpolationResult:
        middle = (self.dataset.x[0] + self.dataset.x[-1]) / 2
        if x_value <= middle:
            return self.evaluate_forward(x_value)
        return self.evaluate_backward(x_value)

    def evaluate_forward(self, x_value: float) -> InterpolationResult:
        h = self.dataset.step
        assert h is not None
        t = (x_value - self.dataset.x[0]) / h
        total = self.dataset.y[0]
        terms = [f"y0={total:.8f}"]
        for order in range(1, len(self.dataset.x)):
            coefficient = prod(t - shift for shift in range(order)) / factorial(order)
            difference = self.differences.value(order, 0)
            total += coefficient * difference
            terms.append(f"k={order}: {coefficient:.8f}*{difference:.8f}")
        return InterpolationResult("первая формула Ньютона", total, f"t={t:.8f}; " + "; ".join(terms))

    def evaluate_backward(self, x_value: float) -> InterpolationResult:
        h = self.dataset.step
        assert h is not None
        n = len(self.dataset.x) - 1
        t = (x_value - self.dataset.x[-1]) / h
        total = self.dataset.y[-1]
        terms = [f"yn={total:.8f}"]
        for order in range(1, len(self.dataset.x)):
            coefficient = prod(t + shift for shift in range(order)) / factorial(order)
            difference = self.differences.value(order, n - order)
            total += coefficient * difference
            terms.append(f"k={order}: {coefficient:.8f}*{difference:.8f}")
        return InterpolationResult("вторая формула Ньютона", total, f"t={t:.8f}; " + "; ".join(terms))

```

```

from math import factorial, prod

from utils.dataset import InterpolationDataset
from utils.differences import FiniteDifferenceTable
from .InterpolationResult import InterpolationResult

class StirlingInterpolator:
    def __init__(self, dataset: InterpolationDataset, differences: FiniteDifferenceTable) -> None:
        self.dataset = dataset
        self.differences = differences
        if dataset.step is None:
            raise ValueError("формула Стирлинга требует равноотстоящие узлы")
        if len(dataset.x) < 3 or len(dataset.x) % 2 == 0:
            raise ValueError("формула Стирлинга строится по нечетному числу узлов")

    def evaluate(self, x_value: float) -> InterpolationResult:
        h = self.dataset.step
        assert h is not None
        center = len(self.dataset.x) // 2
        t = (x_value - self.dataset.x[center]) / h
        total = self.dataset.y[center]
        terms = [f"y0={total:.8f}"]

        for order in range(1, len(self.dataset.x)):
            if order % 2 == 1:
                part = self._odd_term(order, center, t)
            else:
                part = self._even_term(order, center, t)
            if part is None:
                break
            coefficient, difference = part
            total += coefficient * difference
            terms.append(f"k={order}: {coefficient:.8f}*{difference:.8f}")

        return InterpolationResult("формула Стирлинга", total, f"a=x{center}, t={t:.8f}; " + "; ".join(terms))

    def _odd_term(self, order: int, center: int, t: float) -> tuple[float, float] | None:
        m = (order + 1) // 2

```

```

left_index = center - m
right_index = center - m + 1
if not self._has_difference(order, left_index) or not self._has_difference(order, right_index):
    return None
coefficient = t * prod(t * t - j * j for j in range(1, m)) / factorial(order)
difference = (self.differences.value(order, left_index) + self.differences.value(order, right_index)) / 2
return coefficient, difference

def _even_term(self, order: int, center: int, t: float) -> tuple[float, float] | None:
    m = order // 2
    diff_index = center - m
    if not self._has_difference(order, diff_index):
        return None
    coefficient = t * t * prod(t * t - j * j for j in range(1, m)) / factorial(order)
    difference = self.differences.value(order, diff_index)
    return coefficient, difference

def _has_difference(self, order: int, index: int) -> bool:
    return 0 <= order < len(self.differences.levels) and 0 <= index < len(self.differences.levels[order])

```

```

from math import factorial, prod

from utils.dataset import InterpolationDataset
from utils.differences import FiniteDifferenceTable
from .InterpolationResult import InterpolationResult

class BesselInterpolator:
    def __init__(self, dataset: InterpolationDataset, differences: FiniteDifferenceTable) -> None:
        self.dataset = dataset
        self.differences = differences
        if dataset.step is None:
            raise ValueError("формула Бесселя требует равноотстоящие узлы")
        if len(dataset.x) < 4:
            raise ValueError("для формулы Бесселя нужно не менее четырех узлов")

    def evaluate(self, x_value: float) -> InterpolationResult:
        h = self.dataset.step
        assert h is not None
        base = self._base_index(x_value)
        t = (x_value - self.dataset.x[base]) / h
        total = (self.dataset.y[base] + self.dataset.y[base + 1]) / 2
        terms = [f"(y0+y1)/2={total:.8f}"]

        for order in range(1, len(self.dataset.x)):
            if order % 2 == 1:
                part = self._odd_term(order, base, t)
            else:
                part = self._even_term(order, base, t)
            if part is None:
                break
            coefficient, difference = part
            total += coefficient * difference
            terms.append(f"k={order}: {coefficient:.8f}*{difference:.8f}")

        return InterpolationResult("формула Бесселя", total, f"x0=x{base}, t={t:.8f}; " + "; ".join(terms))

    def _base_index(self, x_value: float) -> int:
        candidates = range(len(self.dataset.x) - 1)
        return max(
            candidates,
            key=lambda index: (self._available_terms(index), -abs(self._t_for(index, x_value) - 0.5)),
        )

    def _available_terms(self, base: int) -> int:
        count = 0
        for order in range(1, len(self.dataset.x)):
            part = self._odd_term(order, base, 0.5) if order % 2 == 1 else self._even_term(order, base, 0.5)
            if part is None:
                break
            count += 1
        return count

    def _t_for(self, base: int, x_value: float) -> float:
        h = self.dataset.step
        assert h is not None
        return (x_value - self.dataset.x[base]) / h

    def _odd_term(self, order: int, base: int, t: float) -> tuple[float, float] | None:
        m = (order - 1) // 2
        diff_index = base - m
        if not self._has_difference(order, diff_index):
            return None
        coefficient = (t - 0.5) * prod((t + j) * (t - j - 1) for j in range(m)) / factorial(order)
        difference = self.differences.value(order, diff_index)
        return coefficient, difference

```

```
def _even_term(self, order: int, base: int, t: float) -> tuple[float, float] | None:
    m = order // 2
    left_index = base - m
    right_index = base - m + 1
    if not self._has_difference(order, left_index) or not self._has_difference(order, right_index):
        return None
    coefficient = prod((t + j) * (t - j - 1) for j in range(m)) / factorial(order)
    difference = (self.differences.value(order, left_index) + self.differences.value(order, right_index)) / 2
    return coefficient, difference

def _has_difference(self, order: int, index: int) -> bool:
    return 0 <= order < len(self.differences.levels) and 0 <= index < len(self.differences.levels[order])
```

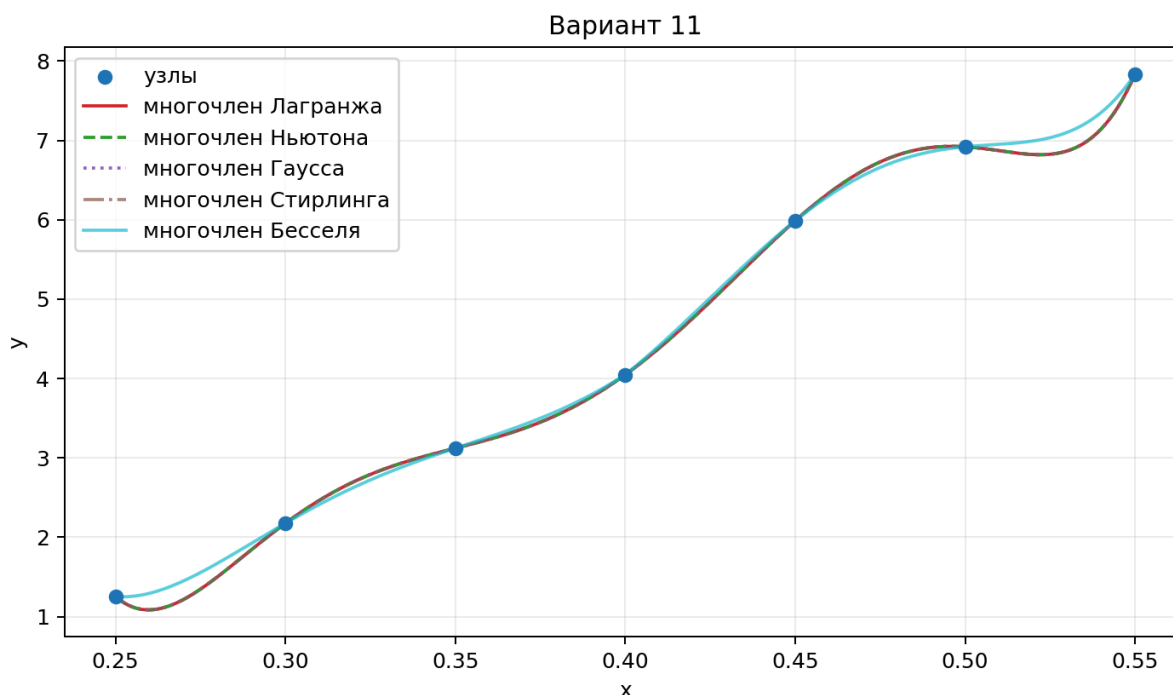
6. Результаты выполнения программы

Для варианта 11 программа построила таблицу конечных разностей и получила следующие значения:

```
x = 0.255
многочлен Лагранжа    1.12252007
первая формула Ньютона 1.12252007
вторая формула Гаусса  1.12252007
формула Стирлинга     1.12252007
формула Бесселя       1.25666336

x = 0.405
многочлен Лагранжа    4.20970802
первая формула Ньютона 4.20970802
вторая формула Гаусса  4.20970802
формула Стирлинга     4.20970802
формула Бесселя       4.22592314
```

График узлов интерполяции и интерполяционных многочленов:



7. Выводы

В ходе лабораторной работы была решена задача интерполяции для варианта 11. Построена таблица конечных разностей, вычислены значения функции в точках $X_1 = 0.255$ и $X_2 = 0.405$ с использованием многочленов Ньютона, Гаусса и Лагранжа. Для дополнительного задания реализованы схемы Стирлинга и Бесселя. Совпадение результатов Стирлинга с полным многочленом подтверждает корректность работы центральной интерполяционной формулы, а схема Бесселя даёт отдельное приближение по чётному числу узлов.